Análise Sintática

Tratamento de Erros em Analisadores Descendentes Recursivos

Erros sintáticos

```
soma := repeat
```

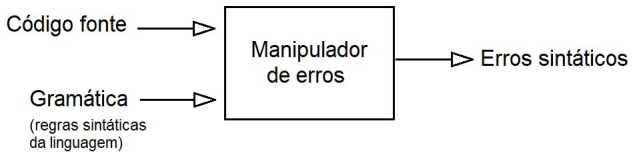
```
read 5
```

```
write .var
```

programa → *decl-seqüência*
decl-seqüência → *declaração* { **;** , *declaração* }
declaração → *if-decl* | *repeat-decl* | *atribuição-decl* | *read-decl* | *write-decl*
if-decl → **if** *exp* **then** *decl-seqüência* [**else** *decl-seqüência*] **end**
repeat-decl → **repeat** *decl-seqüência* **until** *exp*
atribuição-decl → **identificador** **:=** *exp*
read-decl → **read** **identificador**
write-decl → **write** *exp*
exp → *simples-exp* [*comparação-op simples-exp*]
comparação-op → **<** | **=**
simples-exp → *termo* { *soma termo* }
soma → **+** | **-**
termo → *fator* { *mult fator* }
mult → ***** | **/**
fator → **(** *exp* **)** | **número** | **identificador**

Erro sintático

- É uma indicação (ou mensagem) expressa, com relação ao código fonte, de uma inconformidade presente neste referentemente as regras sintáticas da linguagem (expressas em uma gramática), segundo a lógica de um manipulador de erros.



Tratamento de Erros

O mínimo requerido é que um analisador sintático determine se um programa está ou não sintaticamente correto. Um analisador sintático que efetua apenas essa tarefa é denominado **reconhecedor**, pois apenas reconhece as cadeias da linguagem livre de contexto geradas pela gramática da linguagem de programação em questão.

Tratamento de Erros

Além desse comportamento mínimo, um analisador sintático pode apresentar diversos níveis de respostas a erros. Geralmente, um analisador tenta dar uma mensagem significativa de erro, pelo menos para o primeiro erro encontrado.

Tratamento de Erros

- A maioria das técnicas para tratamento de erros é ad hoc.
- Isso ocorre porque se aplicam a linguagens específicas e a algoritmos de análise sintática também específicos os quais contém soluções para situações particulares.
- Não existem regras aplicáveis a todos os casos de tratamento de erros.

Princípios do tratamento de erros

1. Um analisador sintático deve tentar determinar a ocorrência de um erro *tão logo quanto possível*.

2. Após a ocorrência de um erro, o analisador deve escolher um ponto para encerrar o processo. O analisador deve sempre tentar analisar o máximo possível de código, para que o máximo de erros possa ser identificado.

3. Um analisador deve evitar o **problema da cascata de erros**, em que um erro gera uma seqüência de mensagens espúrias.

Tratamento de erros em analisadores descendentes recursivos

Uma forma padrão de recuperação de erros em analisadores descendentes recursivos é denominada **modo de pânico**. O nome advém do fato de que, em situações complexas, o manipulador de erros consome um número possivelmente grande de marcas, ao tentar encontrar um ponto para encerrar a análise sintática (no pior caso, ele pode consumir tudo o que restar do programa, o que é tão ruim quanto simplesmente interromper a execução após o erro). Entretanto, quando bem implementado, este pode ser um método muito bom

Modo Pânico

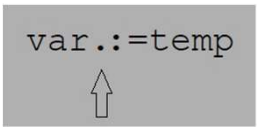
O mecanismo básico do modo de pânico é associar a cada procedimento recursivo um parâmetro adicional composto por um conjunto de **marcas de sincronização**. Com o desenrolar da análise sintática, as marcas que podem ser de sincronização são acrescentadas a esse conjunto cada vez que ocorre uma ativação. Ao encontrar um erro, o analisador **varre à frente**, descartando as marcas até que um dos conjuntos sincronizadores seja encontrado na entrada, quando então a análise é encerrada. As cascatas de erros são evitadas (até certo ponto), pois novas mensagens de erros não são geradas durante a varredura à frente.

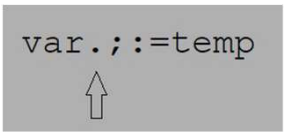
As decisões importantes a tomar nesse método de recuperação de erros são quais marcas acrescentar ao conjunto sincronizador a cada ponto da análise. Geralmente, os conjuntos de Seqüência são candidatos importantes para serem essas marcas sincronizadoras. Os conjuntos Primeiros podem também ser utilizados para evitar que o manipulador de erros descarte marcas importantes que indicam novas construções (como declarações ou expressões).

Gramática de exemplo

<read> → read identificador
<write> → write identificador
<atribuição> → identificador := **<exp aritmética>**
<exp aritmética> → **<operando>** {**<op aritmético>** **<operando>**}
<operando> → identificador | num
<op aritmético> → + | *

Conjunto Sequência





- Caso não haja match:
  - Uma solução plausível é solicitar o próximo token ao analisador léxico até que haja:
    - Match com a marca esperada (ie: ":=")
    - O fim da linha.
  - Esta lógica é baseada na ideia de que houve a inserção de um ou mais tokens inesperados, mas que na sequência os tokens estão corretos (conforme a regra da GLC).

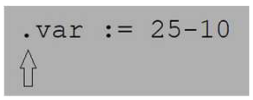
Conjunto Sequência

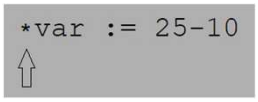
- Composto por todos os tokens (terminais) que sucedem um determinado símbolo da gramática (seja ele variável ou terminal).
- No tratamento de erros, o interesse é no conjunto sequência apenas de símbolos terminais, pois são eles que estão no código fonte sendo analisado.

<read> → read identificador
<write> → write identificador
<atribuição> → identificador := **<exp aritmética>**
<exp aritmética> → **<operando>** {**<op aritmético>** **<operando>**}
<operando> → identificador | num
<op aritmético> → + | *

SEQUÊNCIA (read) = {identificador}
SEQUÊNCIA (write) = {identificador}
SEQUÊNCIA (:=) = {identificador, num}
SEQUÊNCIA (identificador) = {+,*, :=, $}
SEQUÊNCIA (num) = {+, *, $}
SEQUÊNCIA (+) = {identificador, num}
SEQUÊNCIA (*) = {identificador, num}

Conjunto Primeiros





- Caso não haja match:
  - Uma abordagem plausível é considerar tokens que iniciam **instruções** como sendo de sincronização.

Regra que gera instrução da linguagem	Regra de GLC que não gera instrução da linguagem
<atribuição> → identificador := **<exp aritmética>**	**<exp aritmética>** → **<operando>** {**<op aritmético>** **<operando>**}

  - O conjunto Primeiros dos símbolos variáveis são relevantes porque alguns destes símbolos variáveis especificam instruções da linguagem.
  - Para estes símbolos variáveis, interessa saber quais são todos os terminais que podem iniciar estas instruções, ou seja, o conjunto primeiros destes variáveis.

Conjunto Primeiros

- O conjunto primeiros (de um símbolo variável) contém todos os terminais que iniciam formas sentenciais derivadas por aquele símbolo variável.
- No tratamento de erros o interesse é no conjunto primeiro de símbolos variáveis que geram as **instruções** da linguagem.

<read> → read identificador
<write> → write identificador
<atribuição> → identificador := **<exp aritmética>**
<exp aritmética> → **<operando>** {**<op aritmético>** **<operando>**}
<operando> → identificador | num
<op aritmético> → + | *

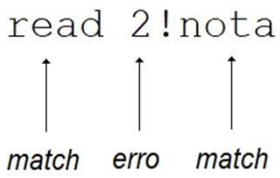
PRIMEIROS(<read>) = {read}
PRIMEIROS(<write>) = {write}
PRIMEIROS(<atribuição>) = {identificador}

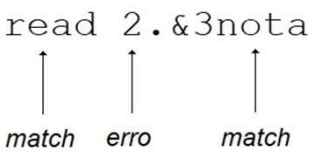
Gramática de exemplo

Considere a seguinte gramática para exemplificar a aplicação do modo pânico:

<read> → read identificador
<write> → write identificador
<atribuição> → identificador := **<exp aritmética>**
<exp aritmética> → **<operando>** {**<op aritmético>** **<operando>**}
<operando> → identificador | num
<op aritmético> → + | *

Exemplos – Modo pânico:



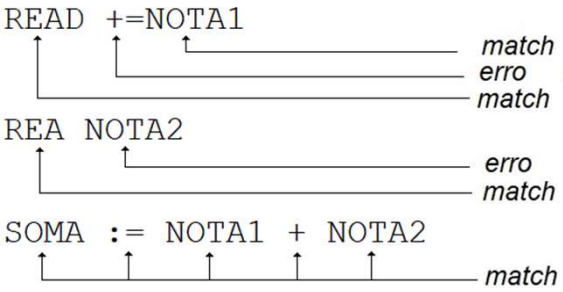


Procedimento do Modo pânico:

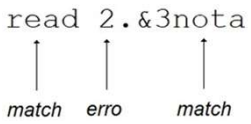
```
Para cada linha do código
   Pedir o próximo token ao analisador léxico
       Se o token for o esperado
            Faz o MATCH
       Senão
            Se for o primeiro token após o último MATCH
               ou for o primeiro token da linha
                  Emite msg de erro
            Varre a frente até token de sincronização
```

Exemplo - Modo pânico

```
READ NOTA1
READ NOTA2
SOMA := NOTA1 + NOTA2
```



Como o Modo pânico implementa os princípios de tratamento de erros:



- Um analisador sintático deve tentar determinar a ocorrência de um erro tão logo quanto possível.
  - *Emite mensagem de erro para todo primeiro token que não case com o esperado após o último match*
- Após a ocorrência de um erro, o analisador deve escolher um ponto para encerrar o processo. O analisador deve sempre tentar analisar o máximo possível de código, para que o máximo de erros possa ser identificado.
  - *Para cada ocorrência de não casamento da marca lida com a marca esperada o analisador emite mensagem de erro e varre à frente em busca de uma marca de sincronização*
- Um analisador deve evitar o problema da cascata de erros, em que um erro gera uma sequência de mensagens espúrias.
  - *Após emitir uma mensagem de erro, o analisador varre à frente e não emite novas mensagens de erro até que haja um match. A partir deste ponto uma nova mensagem de erro pode ser emitida caso não haja um casamento entre o token lido e o esperado.*